

June 8, 2026

Policy-Carrying Fields for Application-Specific Security Tooling

An argument for making security intent concrete on ordinary ORM fields, then using generated metadata and static analysis to enforce local invariants.

Proof-Driven AppSec Secure Codegen Authorization Evidence SAST

Abstract

Many security failures happen because code loses track of what a field means. A patient diagnosis is not just text. A webhook secret is not just a string. An internal risk flag is not just a boolean. These fields carry policy. They define who may see the value, who may mutate it, where it may be serialized, whether it may enter logs, and which review evidence is required when the field crosses a boundary.

Most applications already have a place where this meaning can live. They do not need a new ORM. They need policy metadata on the ORM or schema layer they already use, plus tooling that can read that metadata and enforce local invariants.

This paper argues for policy-carrying fields as a practical foundation for application-specific security tooling. A field annotation makes intent concrete. Generated metadata makes the intent inspectable. Static analysis connects the field to serializers, routes, services, tasks, logs, and mutation paths. AI-assisted tooling lowers the cost of writing the invariant checks, but the security decision remains grounded in generated metadata and reviewable evidence.

The motivating implementation pattern is ordinary model code with explicit policy metadata. Put policy where the application defines data, carry it into

generated metadata, and let enforcement tools verify that code paths obey it. The main argument applies to existing model-based ORMs such as Django.

1. Introduction

Security intent is often discussed in review comments and forgotten in code.

A reviewer says a field should never appear in logs. Another reviewer says a field should be visible to support staff and hidden from customers. A product decision says physicians may edit a treatment plan only under an active care relationship. A privacy decision says an internal risk score may exist in the database and stay out of public APIs.

Those rules are specific, local, and enforceable. The problem is that they are usually written outside the code path that moves the data.

Generic scanners can detect common dangerous APIs and familiar vulnerability classes. They are much weaker at understanding that this application treats one field as physician-visible PHI, another as staff-only operational metadata, and another as safe for redacted logging. The missing layer is a machine-readable declaration of field meaning.

Policy-carrying fields provide that layer. The field definition stays inside the ordinary schema or ORM system. The application continues to use its normal model layer. The difference is that fields carry annotations the toolchain can inspect.

In healthcare, that might mean a field is marked as PHI, physician-visible, nurse-visible, patient-visible, log-redacted, never-log, audit-required, or mutation-restricted. In fintech, the same pattern might apply to account numbers, transaction metadata, sanctions screening results, fraud flags, and internal risk reasons. The domain changes. The mechanism stays the same.

2. Add Policy to the Existing ORM

The practical version of this idea does not replace the ORM. It adds policy to the ORM surface the application already trusts.

In Django, Flask-SQLAlchemy, Rails, and similar frameworks, that surface is usually the model definition. The model already defines field type, nullability, relationships, indexes, defaults, and validation hints. Policy metadata belongs beside those properties because the field is where the application gives data its durable shape.

The shape is simple. A normal Django model field can be wrapped by a policy-aware field class or extended with metadata the application already expects tooling to understand.

```
class PatientNote(models.Model):
    patient = models.ForeignKey(Patient, on_delete=models.CASCADE)

    diagnosis = PolicyTextField(
        policy=[PHI, PhysicianSees, LogRedacted],
    )

    treatment_plan = PolicyTextField(
        policy=[PHI, PhysicianSees, PhysicianMutates, AuditRequired],
    )

    internal_risk_flag = PolicyCharField(
        max_length=64,
        null=True,
        policy=[StaffOnly, NeverLog],
    )
```

This is not a new persistence model. The application still reads and writes normal model objects. The annotations give tools enough information to ask whether access, mutation, serialization, and logging paths respect the declared policy.

The field class can expose metadata to static analyzers, serializers, logging checks, test fixtures, and CI rules. In Django, the metadata can live on the field object, in `Meta`, in a companion registry, or in generated model metadata. The exact storage mechanism matters less than making the policy inspectable from the same model surface developers already use.

The important lesson is that model-level field policy can become executable security infrastructure when the analyzer understands it.

3. Intent Becomes Concrete

Field annotations are not proof. They are claims the system can inspect.

That distinction matters. A field marked `NeverLog` may still be logged by a careless code path. A field marked `PhysicianMutates` may still be modified through an unguarded service. A field marked `OwnerSees` may still leak through an overly broad serializer. The annotation does not make the implementation safe. It gives enforcement tooling a concrete claim to verify.

Without the annotation, the reviewer has to infer policy from naming, domain memory, and nearby code. With the annotation, the question becomes sharper. Does this field enter a log sink. Does this serializer include a field the audience cannot see. Does this mutation path prove the actor has the required relationship. Does this query scope bind the patient, physician, and care context before the sensitive effect.

The annotation turns review intent into a stable surface. The analyzer can read it repeatedly. The test suite can fixture it. CI can enforce it after the check

has earned trust. A human reviewer can inspect the generated evidence and challenge the reasoning.

For healthcare, the field is often the correct level of precision. A model may contain PHI and non-PHI. It may contain data visible to patients, physicians, billing staff, and internal reviewers. It may contain fields that can be logged after redaction and fields that should never be logged at all. Model-level traits are useful for broad categories, but field-level annotations capture the policy that decides where data may flow.

4. The Compiler Shape

Application-specific security tooling does not need mystical analysis. Much of the work is ordinary compiler and graph work applied to application code.

The analyzer reads the schema or model layer and extracts policy-bearing fields. It reads routes, handlers, service methods, serializers, background jobs, logging calls, and mutation sinks. It builds facts about where fields are loaded, transformed, checked, serialized, logged, or written. Then it connects those facts into a graph that can be queried.

Tarjan's algorithm is a useful example because real applications are not clean trees. Service methods call helpers, helpers call other services, serializers call methods, and framework hooks create cycles. Strongly connected components let the analyzer collapse cyclic regions of the call graph into units it can reason about. The result is not perfect understanding. It is a tractable structure for asking local questions.

For a `NeverLog` field, the question is whether any path from the field reaches a logging sink without an approved suppression or redaction step. For a `LogRedacted` field, the question is whether the path passes through an approved redaction helper before logging. For a `PhysicianMutates` field, the question is whether a mutation path has evidence of active treatment relationship, write-scoped role, and audit emission before the write.

AI helps because building these analyzers is repetitive. It can draft the AST extractor, generate graph fixtures, implement the first query, inspect false positives, and turn examples into tests. The useful endpoint is still a versioned control the repository can run again.

5. Logging Is the Hard Test

Logging is a good test for policy-carrying fields because logging often defeats vague security intent.

Developers rarely intend to leak sensitive data. They log request objects, error contexts, event payloads, serializer outputs, model snapshots, and diagnostic maps because the system is failing and they need evidence. If field policy is stored only in a wiki page, logging paths will drift. If the policy is carried by the field, tooling can inspect whether the field reaches a log sink.

The enforcement rule can be narrow. Fields marked `NeverLog` may not reach logging sinks. Fields marked `LogRedacted` may reach logging sinks only through approved redaction helpers. Fields marked `LogAllowed` may be logged directly, with the finding artifact still showing the path.

This is where application-specific controls beat generic scanners. A generic scanner can warn about logging a variable named `password` or `token`. A local invariant checker can know that `diagnosis`, `internal_risk_flag`, `recovery_context`, or `payer_denial_reason` carries a specific policy even when the name is not obviously sensitive to a universal tool.

The artifact should be reviewable. It should name the field, the declared policy, the source path, the sink, the transformations seen on the path, and the missing proof. The reviewer should not have to trust a model's summary. The claim should terminate in source-backed evidence.

6. Generated Metadata as a Concrete Pattern

The model annotation is only the beginning. The useful control comes from carrying the field policy into generated metadata that analyzers can read consistently.

In a Django-style implementation, each policy-bearing field can expose metadata describing visibility, mutation authority, logging behavior, serialization audience, audit requirements, and storage requirements. A checker can read that metadata directly from the model layer and compare it against serializers, log sinks, query scopes, and mutation paths.

Generated metadata gives security tooling a stable place to stand. A sensitive field routed into a public serializer can be flagged. A field marked `NeverLog` can be traced toward logging sinks. A field marked `PhysicianMutates` can require evidence of relationship, role, workflow, and audit before a write path is accepted.

The healthcare extension is a natural next step. A model field can declare healthcare-specific policy such as PHI, physician visibility, mutation authority, log redaction, audit requirements, and patient-facing serialization. The generated model metadata carries those policies forward. Static analysis then checks whether application code respects them.

Pickle is a useful reference point because it already treats schema declarations as the source of generated model metadata, response types, serialization helpers, ownership scopes, and visibility maps. Its migration annotations are one way to provide a faithful policy surface. The same idea can be applied to an existing ORM by augmenting normal model fields rather than replacing the model layer.

7. Failure Modes

Policy-carrying fields can fail if the annotations become decoration.

The first failure mode is assertion without verification. A field marked `NeverLog` means little if no tool checks logging paths. The annotation has to feed a checker, a fixture, a review artifact, or a CI gate.

The second failure mode is policy drift. A field may start as staff-only and later become safe for patient-facing export. Another field may start as safe for logs and later become sensitive after product changes. Annotations need owners, review history, and tests.

The third failure mode is broad annotation vocabulary. If every team invents twenty overlapping labels, the tool cannot enforce them consistently. The first vocabulary should be small. It should focus on visibility, mutation authority, logging, serialization audience, encryption, and audit.

The fourth failure mode is false precision. A field-level policy is precise, but field flow is still hard. Dynamic dispatch, reflection, framework callbacks, and serialization helpers can hide paths. A useful tool reports unresolved evidence when it cannot prove safety.

The fifth failure mode is treating AI output as proof. AI can help write the checker, but the checker has to run against source, produce evidence, and survive review without the original agent session.

8. Implementation Sketch

Start with one policy family.

For healthcare, logging is a strong pilot. Add `NeverLog`, `LogRedacted`, and `LogAllowed` to a small set of fields that already cause review concern.

Generate metadata from the schema layer. Write a checker that follows those

fields into serializers, event payloads, error contexts, and logging calls. Treat early findings as review candidates.

Once the logging check produces low noise, add visibility. Fields can declare physician-visible, patient-visible, support-visible, or staff-only policy. The checker can compare field policy against serializers and query scopes. The finding artifact should show the field, audience, serializer or route, and missing proof.

Mutation authority should come later. It is usually harder than visibility because it depends on actor state, relationship, workflow, and audit. A `PhysicianMutates` field should require evidence of active treatment relationship, write-scoped role, and audit emission. The first checker should be narrow enough that reviewers can understand every finding.

The final step is promotion. Signals become warnings after the team understands the false positives. Warnings become lintable errors after the check repeatedly finds real drift. CI enforcement should come only after the control has been run across enough code and history to earn trust.

9. Conclusion

Application-specific security tooling gets stronger when applications make their own policy visible.

Field annotations are a practical way to do that. They attach security meaning to ordinary schema fields, allow generated code to retain that meaning, and give analyzers a stable surface for enforcement. In domains such as healthcare, where field-level meaning determines visibility, mutation authority, logging safety, and audit requirements, that precision matters.

The shape is straightforward. Put policy on the model or schema surface. Generate metadata from it. Use static analysis to connect field policy to code paths. Use AI to lower the cost of writing and refining the enforcement checks.

Keep the final authority in reviewable artifacts, tests, and repository-owned controls.

The result is not a new ORM. It is a way to make the ORM the application already uses carry enough security intent for tooling to enforce the rules that matter locally.